

最后请注意，STL 容器所使用的 heap 内存是由容器所拥有的分配器对象（allocator objects）管理，不是被 new 和 delete 直接管理。本章并不讨论 STL 分配器。

条款 49：了解 new-handler 的行为

Understand the behavior of the new-handler.

当 operator new 无法满足某一内存分配需求时，它会抛出异常。以前它会返回一个 null 指针，某些旧式编译器目前也还那么做。你还是可以取得旧行为（有那么几分像啦），但本条款最后才会进行这项讨论。

当 operator new 抛出异常以反映一个未获满足的内存需求之前，它会先调用一个客户指定的错误处理函数，一个所谓的 *new-handler*。（这其实并非全部事实。operator new 真正做的事情稍微更复杂些。详见条款 51。）为了指定这个“用以处理内存不足”的函数，客户必须调用 set_new_handler，那是声明于 <new> 的一个标准程序库函数：

```
namespace std {  
    typedef void (*new_handler)();  
    new_handler set_new_handler(new_handler p) throw();  
}
```

如你所见，new_handler 是个 typedef，定义出一个指针指向函数，该函数没有参数也不返回任何东西。set_new_handler 则是“获得一个 new_handler 并返回一个 new_handler”的函数。set_new_handler 声明式尾端的 “throw()” 是一份异常明细，表示该函数不抛出任何异常——虽然事实更有趣些，详见条款 29。

set_new_handler 的参数是个指针，指向 operator new 无法分配足够内存时该被调用的函数。其返回值也是个指针，指向 set_new_handler 被调用前正在执行（但马上就要被替换）的那个 new-handler 函数。

你可以这样使用 set_new_handler：

```
//以下是当 operator new 无法分配足够内存时，该被调用的函数  
void outOfMem()  
{  
    std::cerr << "Unable to satisfy request for memory\n";  
    std::abort();  
}
```

```
int main()
{
    std::set_new_handler(outOfMem);
    int* pBigDataArray = new int[100000000L];
    ...
}
```

就本例而言, 如果 `operator new` 无法为 100,000,000 个整数分配足够空间, `outOfMem` 会被调用, 于是程序在发出一个信息之后夭折 (`abort`)。(顺带一提, 如果在写出错误信息至 `cerr` 过程期间必须动态分配内存, 考虑会发生什么事……)

当 `operator new` 无法满足内存申请时, 它会不断调用 `new-handler` 函数, 直到找到足够内存。引起反复调用的代码显示于条款 51, 这里的高级描述已足够获得一个结论, 那就是一个设计良好的 `new-handler` 函数必须做以下事情:

- **让更多内存可被使用。**这便造成 `operator new` 内的下一次内存分配动作可能成功。实现此策略的一个做法是, 程序一开始执行就分配一大块内存, 而后当 `new-handler` 第一次被调用, 将它们释还给程序使用。
- **安装另一个 `new-handler`。**如果目前这个 `new-handler` 无法取得更多可用内存, 或许它知道另外哪个 `new-handler` 有此能力。果真如此, 目前这个 `new-handler` 就可以安装另外那个 `new-handler` 以替换自己 (只要调用 `set_new_handler`)。下次当 `operator new` 调用 `new-handler`, 调用的将是最新安装的那个。(这个旋律的变奏之一是让 `new-handler` 修改自己的行为, 于是当它下次被调用, 就会做某些不同的事。为达此目的, 做法之一是令 `new-handler` 修改“会影响 `new-handler` 行为”的 `static` 数据、`namespace` 数据或 `global` 数据。)
- **卸除 `new-handler`，**也就是将 `null` 指针传给 `set_new_handler`。一旦没有安装任何 `new-handler`, `operator new` 会在内存分配不成功时抛出异常。
- **抛出 `bad_alloc` (或派生自 `bad_alloc`) 的异常。**这样的异常不会被 `operator new` 捕捉, 因此会被传播到内存索求处。
- **不返回, 通常调用 `abort` 或 `exit`。**

这些选择让你在实现 `new-handler` 函数时拥有很大弹性。

有时候你或许希望以不同的方式处理内存分配失败情况，你希望视被分配物属于哪个 class 而定：

```
class X {
public:
    static void outOfMemory();
    ...
};

class Y {
public:
    static void outOfMemory();
    ...
};

X * p1 = new X;    //如果分配不成功,
                  //调用 X::outOfMemory
Y * p2 = new Y;    //如果分配不成功,
                  //调用 Y::outOfMemory
```

C++ 并不支持 class 专属之 new-handlers，但其实也不需要。你可以自己实现出这种行为。只需令每一个 class 提供自己的 set_new_handler 和 operator new 即可。其中 set_new_handler 使客户得以指定 class 专属的 new-handler（就像标准的 set_new_handler 允许客户指定 global new-handler），至于 operator new 则确保在分配 class 对象内存的过程中以 class 专属之 new-handler 替换 global new-handler。

现在，假设你打算处理 Widget class 的内存分配失败情况。首先你必须登录“当 operator new 无法为一个 Widget 对象分配足够内存时”调用的函数，所以你需要声明一个类型为 new_handler 的 static 成员，用以指向 class Widget 的 new-handler。看起来像这样：

```
class Widget {
public:
    static std::new_handler set_new_handler(std::new_handler p) throw();
    static void* operator new(std::size_t size) throw(std::bad_alloc);
private:
    static std::new_handler currentHandler;
};
```

Static 成员必须在 class 定义式之外被定义（除非它们是 const 而且是整数型，见条款 2），所以需要这么写：

```
std::new_handler Widget::currentHandler = 0;
//在 class 实现文件内初始化为 null
```

Widget 内的 `set_new_handler` 函数会将它获得的指针存储起来, 然后返回先前 (在此调用之前) 存储的指针, 这也正是标准版 `set_new_handler` 的作为:

```
std::new_handler Widget::set_new_handler(std::new_handler p) throw()
{
    std::new_handler oldHandler = currentHandler;
    currentHandler = p;
    return oldHandler;
}
```

最后, Widget 的 `operator new` 做以下事情:

1. 调用标准 `set_new_handler`, 告知 Widget 的错误处理函数。这会将 Widget 的 `new-handler` 安装为 `global new-handler`。
2. 调用 `global operator new`, 执行实际之内存分配。如果分配失败, `global operator new` 会调用 Widget 的 `new-handler`, 因为那个函数才刚被安装为 `global new-handler`。如果 `global operator new` 最终无法分配足够内存, 会抛出一个 `bad_alloc` 异常。在此情况下 Widget 的 `operator new` 必须恢复原本的 `global new-handler`, 然后再传播该异常。为确保原本的 `new-handler` 总是能够被重新安装回去, Widget 将 `global new-handler` 视为资源并遵守条款 13 的忠告, 运用资源管理对象 (resource-managing objects) 防止资源泄漏。
3. 如果 `global operator new` 能够分配足够一个 Widget 对象所用的内存, Widget 的 `operator new` 会返回一个指针, 指向分配所得。Widget 析构函数会管理 `global new-handler`, 它会自动将 Widget's `operator new` 被调用前的那个 `global new-handler` 恢复回来。

下面以 C++ 代码再阐述一次。我将从资源处理类 (resource-handling class) 开始, 那里面只有基础性 `RAII` 操作, 在构造过程中获得一笔资源, 并在析构过程中释还 (见条款 13):

```
class NewHandlerHolder {
public:
    explicit NewHandlerHolder(std::new_handler nh)    //取得目前的
        : handler(nh) {}                             //new-handler.

    ~NewHandlerHolder()                               //释放它
    { std::set_new_handler(handler); }

private:
    std::new_handler handler;                         //记录下来.

    NewHandlerHolder(const NewHandlerHolder&);        //阻止 copying
    NewHandlerHolder&
        operator=(const NewHandlerHolder&);          //(见条款 14)
};
```

这就使得 Widget's operator new 的实现相当简单:

```
void* Widget::operator new(std::size_t size) throw(std::bad_alloc)
{
    NewHandlerHolder                                //安装 Widget 的
    h(std::set_new_handler(currentHandler));         //new-handler.
    return ::operator new(size);                     //分配内存或抛出异常.
}                                                    //恢复 global new-handler.
```

Widget 的客户应该类似这样使用其 new-handling:

```
void outOfMem();                                     //函数声明。此函数在
                                                    //Widget 对象分配失败时被调用.

Widget::set_new_handler(outOfMem);                  //设定 outOfMem 为 Widget 的
                                                    // new-handling 函数.

Widget* pw1 = new Widget;                           //如果内存分配失败,
                                                    //调用 outOfMem.

std::string* ps = new std::string;                  //如果内存分配失败,
                                                    //调用 global new-handling 函数
                                                    //(如果有的话).

Widget::set_new_handler(0);                          //设定 Widget 专属的
                                                    //new-handling 函数为 null.

Widget* pw2 = new Widget;                           //如果内存分配失败,
                                                    //立刻抛出异常.
                                                    //(class Widget 并没有专属的
                                                    //new-handling 函数).
```

实现这一方案的代码并不因 class 的不同而不同, 因此在它处加以复用是个合理的构想。一个简单的做法是建立起一个 "mixin" 风格的 base class, 这种 base class 用来允许 derived classes 继承单一特定能力——在本例中是“设定 class 专属之 new-handler”的能力。然后将这个 base class 转换为 template, 如此一来每个 derived class 将获得实体互异的 class data 复件。

这个设计的 base class 部分让 derived classes 继承它们所需的 set_new_handler 和 operator new, 而 template 部分则确保每一个 derived class 获得一个实体互异的 currentHandler 成员变量。听起来似乎有点复杂, 但代码非常近似前个版本。实际上, 唯一真正意义上的不同是, 它现在可被任何有所需要的 class 使用:

```

template<typename T>                // "mixin" 风格的 base class, 用以支持
class NewHandlerSupport {           // class 专属的 set_new_handler
public:
    static std::new_handler set_new_handler(std::new_handler p) throw();
    static void* operator new(std::size_t size) throw(std::bad_alloc);
    ...                             // 其他的 operator new 版本——见条款 52

private:
    static std::new_handler currentHandler;
};

template<typename T>
std::new_handler
NewHandlerSupport<T>::set_new_handler(std::new_handler p) throw()
{
    std::new_handler oldHandler = currentHandler;
    currentHandler = p;
    return oldHandler;
}

template<typename T>
void* NewHandlerSupport<T>::operator new(std::size_t size)
throw(std::bad_alloc)
{
    NewHandlerHolder h(std::set_new_handler(currentHandler));
    return ::operator new(size);
}

// 以下将每一个 currentHandler 初始化为 null
template<typename T>
std::new_handler NewHandlerSupport<T>::currentHandler = 0;

```

有了这个 **class template**, 为 Widget 添加 set_new_handler 支持能力就轻而易举了: 只要令 Widget 继承自 NewHandlerSupport<Widget> 就好, 像下面这样。看起来似乎很奇妙, 稍后我将更详细解释它的精确意义。

```

class Widget: public NewHandlerSupport<Widget> {
    ...                // 和先前一样, 但不必声明
};                    // set_new_handler 或 operator new

```

这就是 Widget 为了提供“class 专属之 set_new_handler”所需做的全部动作。

但或许你还是对 Widget 继承 NewHandlerSupport<Widget> 感到心慌意乱。果真如此, 你的焦虑还可能因为注意到 NewHandlerSupport **template** 从未使用其类型参数 **T** 而更放大数倍。实际上 **T** 的确不需被使用。我们只是希望, 继承自 NewHandlerSupport 的每一个 class, 拥有实体互异的 NewHandlerSupport 复件 (更明确地说是其 **static** 成员变量 **currentHandler**)。类型参数 **T** 只是用来区分不同的

derived class。Template 机制会自动为每一个 T (NewHandlerSupport 赖以具现化的根据) 生成一份 currentHandler。

至于说到 Widget 继承自一个模板化的 (templated) base class, 而后者又以 Widget 作为类型参数, 如果你对此头昏眼花, 不要觉得惭愧。每个人一开始都有那种反应。由于它被证明是一个有用的技术, 因此甚至拥有自己的名称: “怪异的循环模板模式” (*curiously recurring template pattern*; CRTP)。有些人认为这个名称给人的第一眼印象很不自然。嗯, 确实如此。

我曾发表过一篇文章, 建议给它一个比较好的名称, 像是 Do It For Me, 因为当 Widget 继承 NewHandlerSupport<Widget> 时它其实并不是说 “我是 Widget, 我要针对 Widget class 继承 NewHandlerSupport”。但是, 哎, 没人采用我建议的名称 (甚至我自己也不), 但如果你看到 CRTP 会联想到它说的是 “do it for me”, 或许可以帮助你了解这一模板化继承 (templated inheritance) 到底用意为何。

像 NewHandlerSupport 这样的 templates, 使得 “为任何 class 添加一个它们专属的 new-handler” 成为易事。然而 “mixin” 风格的继承肯定导致多重继承的争议, 而在开始那条路之前, 你需要先阅读条款 40。

直至 1993 年, C++ 都还要求 operator new 必须在无法分配足够内存时返回 null。新一代的 operator new 则应该抛出 bad_alloc 异常, 但很多 C++ 程序是在编译器开始支持新修规范前写出来的。C++ 标准委员会不想抛弃那些 “侦测 null” 的族群, 于是提供另一形式的 operator new, 负责供应传统的 “分配失败便返回 null” 行为。这个形式被称为 “nothrow” 形式——某种程度上是因为他们在 new 的使用场合用了 nothrow 对象 (定义于头文件 <new>) :

```
class Widget { ... };
Widget* pw1 = new Widget;           //如果分配失败, 抛出 bad_alloc.
if (pw1 == 0) ...                   //这个测试一定失败.
Widget* pw2 = new (std::nothrow) Widget; //如果分配 Widget 失败, 返回 0.
if (pw2 == 0) ...                   //这个测试可能成功
```

Nothrow new 对异常的强制保证性并不高。要知道, 表达式 “new (std::nothrow) Widget” 发生两件事, 第一, nothrow 版的 operator new 被调用, 用以分配足够内存给 Widget 对象。如果分配失败便返回 null 指针, 一如文档所言。如果分配成功, 接下来 Widget 构造函数会被调用, 而在那一点上所有的筹码便都耗尽, 因为

Widget 构造函数可以做它想做的任何事。它有可能又 new 一些内存, 而没人可以强迫它再次使用 `nothrow new`。因此虽然 `"new (std::nothrow) Widget"` 调用的 `operator new` 并不抛掷异常, 但 Widget 构造函数却可能会。如果它真那么做, 该异常会一如往常地传播。需要结论吗? 结论就是: 使用 `nothrow new` 只能保证 `operator new` 不抛掷异常, 不保证像 `"new (std::nothrow) Widget"` 这样的表达式绝不导致异常。因此你其实没有运用 `nothrow new` 的需要。

无论使用正常 (会抛出异常) 的 `new`, 或是其多少有点发育不良的 `nothrow` 兄弟, 重要的是你需要了解 `new-handler` 的行为, 因为两种形式都使用它。

请记住

- `set_new_handler` 允许客户指定一个函数, 在内存分配无法获得满足时被调用。
- `Nothrow new` 是一个颇为局限的工具, 因为它只适用于内存分配; 后继的构造函数调用还是可能抛出异常。

条款 50: 了解 new 和 delete 的合理替换时机

Understand when it makes sense to replace new and delete.

让我们暂时回到根本原理。首先, 怎么会有人想要替换编译器提供的 `operator new` 或 `operator delete` 呢? 下面是三个最常见的理由:

- 用来检测运用上的错误。如果将“new 所得内存”delete 掉却不幸失败, 会导致内存泄漏 (memory leaks)。如果在“new 所得内存”身上多次 delete 则会导致不确定行为。如果 `operator new` 持有一串动态分配所得地址, 而 `operator delete` 将地址从中移走, 倒是很容易检测出上述错误用法。此外各式各样的编程错误可能导致数据 “overruns” (写入点在分配区块尾端之后) 或 “underruns” (写入点在分配区块起点之前)。如果我们自行定义一个 `operator new`, 便可超额分配内存, 以额外空间 (位于客户所得区块之前或后) 放置特定的 byte patterns (即签名, signatures)。operator deletes 便得以检查上述签名是否原封不动, 若否就表示在分配区的某个生命时间点发生了 overrun 或 underrun, 这时候 `operator delete` 可以志记 (log) 那个事实以及那个惹是生非的指针。